

总结报告-MXFP8 / NVFP4 低精度模拟与反量化

项目完成人：江昆阳

github ID：YeKcaJ

一. 题目概述与项目背景

概述

任务背景是：为下一代大模型推理引擎开发低精度权重加载模块，需要评估 MXFP8、NVFP4 这两类格式在模型压缩与推理加速中的价值。但现实约束是测试环境**无法提供 Hopper、Blackwell 或更新架构的 GPU**，手上只有常规 CUDA GPU。因此不能依赖 FP8/FP4 Tensor Core，也不能依赖新架构引入的低精度存储与转换指令，必须用常规 CUDA kernel 完成**软件模拟**：低精度编码、打包、缩放、解包、反量化与误差评估全部自己实现。

要求

1. 功能正确性

- 正确实现 MXFP8 数据表示、block scaling 和反量化流程；
- 正确实现 NVFP4 的数值编码、元素局部缩放、全局缩放、打包、解包与反量化规则；
- 正确实现 per-tensor scaling 和 block-wise scaling；
- 对随机矩阵、正态分布矩阵和含异常值矩阵分别输出误差统计。

2. 硬件要求

- 基础版本不得依赖 Hopper、Blackwell 或 Ampere 以上架构特性；
- 基础版本不得要求硬件原生支持 FP8 / FP4 Tensor Core；
- 如有支持 FP8 / FP4 的 GPU，可额外实现硬件或库路径作为加分项，但不能作为基础验收前提；

3. 4 bit 打包与内存访问

- 4 bit 格式必须实现 packed load / packed store；

4. 平台适配

- 要求不止在英伟达平台，其他国产平台亦可

二. 数值格式与算法设计

统一符号与基本思路

设输入矩阵有 R 行、 C 列，总元素数为 N ：

$$X \in \mathbb{R}^{R \times C}, \quad N = R \times C$$

按照行主序展开，即先连续读取第一行：

$$x_i = X_{\lfloor i/C \rfloor, i \bmod C}, \quad 0 \leq i < N$$

FP16 输入先无损扩展为 FP32，后续缩放计算统一在 FP32 中进行。设每组包含 B 个元素，第 b 组的元素下标集合为：

$$\mathcal{I}_b = \{i \mid bB \leq i < \min((b+1)B, N)\}$$

分组数量：

$$K = \left\lceil \frac{N}{B} \right\rceil$$

当前实现中，MXFP8 默认每组 32 个元素，NVFP4 默认每组 16 个元素。分组作用于展开后的连续数组，允许跨越矩阵行边界。

量化的基本思路

先通过 scale 调整数值范围，再选择低精度格式中可表示的数值。

用 Q 表示“生成低精度编码”，D 表示“将编码解码成数值”，则：

$$q_i = Q\left(\frac{x_i}{S_b}\right), \quad \hat{x}_i = D(q_i) \times S_b$$

MXFP8 分组缩放与E4M3编码

1. 确定元素的可表示范围

本项目使用 E4M3FN 编码，每个元素占 8 bit，由 1 位符号、4 位指数、3 位尾数组成：

$$\underbrace{s}_{1 \text{ bit}} \quad \underbrace{e}_{4 \text{ bit}} \quad \underbrace{m}_{3 \text{ bit}}$$

对于正常有限数，解码公式为：

$$D_{\text{E4M3}}(s, e, m) = (-1)^s \left(1 + \frac{m}{8}\right) 2^{e-7}$$

其中，指数偏置为 7。指数为零时，使用非正规数公式：

$$D_{\text{E4M3}}(s, 0, m) = (-1)^s m 2^{-9}$$

指数为 15、尾数为 7 的编码保留为 NaN，不参与有限数编码。因此最大有限幅值为：

$$\left(1 + \frac{6}{8}\right) 2^{15-7} = 448$$

所以，E4M3 元素的有限数值范围为 -448 到 448。

2. 计算组内最大绝对值

对第 b 组计算最大绝对值：

$$M_b = \max_{i \in \mathcal{I}_b} |x_i|$$

采用绝对值，是因为正数和负数具有相同的幅值范围，符号单独保存。这个最大值决定当前分组应该使用多大的 scale。

3. 计算共享 scale

为了让归一化后的最大幅值落入 E4M3 范围，希望满足：

$$\frac{M_b}{S_b} \leq 448 \iff S_b \geq \frac{M_b}{448}$$

MXFP8 的 scale 使用 E8M0 表示。在当前实现中，scale 只能取 2 的整数次幂：

$$S_b = 2^{k_b}$$

因此，在一般非零情况下，选择满足范围要求的最小 2 的幂：

$$k_b = \left\lceil \log_2 \left(\frac{M_b}{448} \right) \right\rceil$$

向上取整的作用，是让 scale 足够大，控制归一化后的数值范围。

4. 将 scale 保存为 E8M0 字节

实际文件中不为每组保存一个 FP32 scale，而是保存带偏置的指数：

$$c_b = k_b + 127$$

读取时，再恢复实际 scale：

$$S_b = 2^{c_b - 127}$$

当前实现允许的 scale 编码范围为 0~254。令 r 为 FP32 运算得到的最大值与 448 的比值，则完整编码规则为：

$$r_b = \text{fl}_{32} \left(\frac{M_b}{448} \right)$$

$$c_b = \begin{cases} 127, & M_b = 0 \\ 0, & M_b > 0 \text{ 且 } r_b = 0 \\ \text{clip}(\lceil \log_2(r_b) \rceil + 127, 0, 254), & \text{其他情况} \end{cases}$$

其中：

- 全零组使用 scale=1。
- 如果除法结果下溢为零，使用最小允许 scale。

- `clip` 表示将数值限制在给定范围内。

5. 归一化并编码元素

首先，将原始元素除以实际 scale：

$$z_i = \frac{x_i}{S_b}$$

然后，将归一化值限制到 E4M3 的有限范围：

$$z_i^{\text{clip}} = \text{clip}(z_i, -448, 448)$$

最后生成 E4M3 编码：

$$q_i = Q_{\text{E4M3}}(z_i^{\text{clip}})$$

6. 反量化

读取元素编码和对应的 scale 编码，计算：

$$\hat{x}_i = D_{\text{E4M3}}(q_i) \times 2^{c_b-127}$$

7. 例子

例如，某组最大绝对值为 100，其中一个元素为10.3

$$\begin{aligned} M_b &= 100 \\ \frac{M_b}{448} &\approx 0.223214 \\ k_b &= \lceil \log_2(0.223214) \rceil = -2 \\ S_b &= 2^{-2} = 0.25 \\ \frac{M_b}{S_b} &= 400 \end{aligned}$$

此时的scale为0.25

$$z_i = \frac{10.3}{0.25} = 41.2$$

41.2 附近的 E4M3 可表示值包括 40 和 44，nearest 舍入选择 40：

$$D_{\text{E4M3}}(q_i) = 40$$

反量化执行，读取元素编码和对应的 scale 编码，计算：

$$\hat{x}_i = 40 \times 0.25 = 10$$

因此，量化引入的绝对误差约为：

$$|x_i - \hat{x}_i| = |10.3 - 10| = 0.3$$

NVFP4 两级缩放与四位打包

1. 明确 E2M1 的数值集合

NVFP4 元素采用 E2M1 编码，由 1 位符号、2 位指数和 1 位尾数组成：

$$\underbrace{s}_{1 \text{ bit}} \quad \underbrace{e}_{2 \text{ bit}} \quad \underbrace{m}_{1 \text{ bit}}$$

正常数的解码公式为：

$$D_{\text{E2M1}}(s, e, m) = (-1)^s \left(1 + \frac{m}{2}\right) 2^{e-1}, \quad e > 0$$

指数为零时：

$$D_{\text{E2M1}}(s, 0, m) = (-1)^s \frac{m}{2}$$

对应的非负幅值只有：

$$\mathcal{V}_4^+ = \{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$$

低 3 位编码	幅值
000	0
001	0.5
010	1
011	1.5
100	2
101	3
110	4
111	6

最高位保存符号，因此元素的有限数值范围为 -6 到 6。

2. 计算全局最大值

首先对整个张量计算最大绝对值：

$$M = \max_{0 \leq i < N} |x_i|$$

NVFP4 使用两级缩放，实际 scale 为：

$$S_b = g \times s_b$$

其中：

- g 是整个张量共享的 FP32 global scale。
- 局部 scale 使用 E4M3 编码，每个分组保存一个。

global scale 负责调整整个张量的量级，block scale 负责适应不同分组的局部范围。

3. 计算global scale

E2M1 元素的最大幅值为 6，E4M3 block scale 的最大值为 448，两者乘积为：2688

因此，一般情况下选择：

$$g = \frac{M}{2688}$$

这样，最大值对应的理想 block scale 就能接近 E4M3 的上限。考虑全零和浮点下溢，当前实现的完整规则为：

$$g = \begin{cases} 1, & M = 0 \\ \max(\text{fl}_{32}(M/2688), 2^{-149}), & M > 0 \end{cases}$$

这里的最小值是最小正 FP32 非正规数，用于避免 global scale 下溢为零。

4. 计算并量化 block scale

计算第 b 组的最大绝对值：

$$M_b = \max_{i \in \mathcal{I}_b} |x_i|$$

希望归一化后的组内最大幅值接近 E2M1 上限 6：

$$\frac{M_b}{g s_b} \approx 6$$

因此，理想的 block scale 为：

$$s_b^* = \frac{M_b}{6g}$$

但是，这个理想值不能直接作为任意 FP32 数保存，而需要编码为 E4M3：

$$c_b = Q_{\text{E4M3}}^{\text{nearest}}(s_b^*)$$

再解码出实际使用的 block scale：

$$s_b = D_{\text{E4M3}}(c_b)$$

元素量化必须使用解码后的 block scale，而不能继续使用未量化的理想值。 否则，量化时使用的 scale 与文件中保存的 scale 不一致，反量化就无法遵循相同规则。

当前实现中，即使元素使用 stochastic 舍入，block scale 仍采用 nearest 舍入。

5. 计算有效scale并编码元素

首先合成实际使用的 scale：

$$S_b = g \times D_{E4M3}(c_b)$$

然后计算归一化值：

$$z_i = \begin{cases} x_i/S_b, & S_b > 0 \\ 0, & S_b = 0 \end{cases}$$

最后限制范围并编码为 E2M1：

$$q_i = Q_{E2M1}(\text{clip}(z_i, -6, 6))$$

零 scale 分支用于避免除零。全零分组会得到零 block scale；如果某组相对全局范围极小，其 block scale 也可能被量化为零。另外，block scale 自身经过舍入后，可能小于理想值，使部分归一化元素超过 6，因此仍需执行饱和处理。

6. 将两个四位编码打包成一个字节

对于相邻两个元素：

- 偶数下标元素保存到低 4 bit。
- 奇数下标元素保存到高 4 bit。

打包公式为：

$$p_j = (q_{2j} \& 0x0F) | ((q_{2j+1} \& 0x0F) \ll 4)$$

其中， $\&$ 表示按位与， $|$ 表示按位或， $\ll$ 表示左移。

因此，packed data 所需字节数为：

$$L_{\text{data}} = \left\lceil \frac{N}{2} \right\rceil$$

如果元素数量为奇数，最后一个字节的高四位补零。

7. 解包并反量化

第 i 个元素所在字节下标为：

$$j = \left\lfloor \frac{i}{2} \right\rfloor$$

通过移位和掩码取出四位编码：

$$q_i = (p_j \gg (4(i \bmod 2))) \& 0x0F$$

然后解码并恢复数值：

$$\hat{x}_i = D_{E2M1}(q_i) \times (g \times D_{E4M3}(c_b))$$

当前代码先以 FP32 计算括号中的有效 scale，再与元素解码值相乘。CPU/GPU 对照时需要保持相同的计算顺序。

8. 例子

假设整个张量最大绝对值为 5376，其中某个分组的最大绝对值为 12：

$$\begin{aligned} M &= 5376 \\ g &= \frac{5376}{2688} = 2 \\ M_b &= 12 \\ s_b^* &= \frac{12}{6 \times 2} = 1 \\ s_b &= 1 \\ S_b &= 2 \times 1 = 2 \end{aligned}$$

因为 1 可以被 E4M3 精确表示，所以本例中 block scale 没有额外舍入误差。
假设该组中两个相邻元素为 2.6 和 -4：

$$\begin{aligned} x_0 &= 2.6, & x_1 &= -4 \\ z_0 &= \frac{2.6}{2} = 1.3, & z_1 &= \frac{-4}{2} = -2 \end{aligned}$$

nearest 舍入后：

$$D_{E2M1}(q_0) = 1.5, \quad D_{E2M1}(q_1) = -2$$

对应编码分别为 `0x3` 和 `0xC`，打包结果为：

$$p_0 = 0x3 \mid (0xC \ll 4) = 0xC3$$

反量化得到：

$$\begin{aligned} \hat{x}_0 &= 1.5 \times 2 = 3 \\ \hat{x}_1 &= -2 \times 2 = -4 \end{aligned}$$

两个元素的低精度编码共占一个字节，scale 的存储开销由同组元素共同承担。

误差与压缩率

1. 计算最终输出误差

误差比较的是原始输入与实际输出精度下的最终数值，因此包含低精度量化损失和输出类型转换带来的影响。对于非空张量，且所有输出均为有限值：

$$\text{MaxAbsError} = \max_{0 \leq i < N} |x_i - y_i|$$

$$\text{MAE} = \frac{1}{N} \sum_{i=0}^{N-1} |x_i - y_i|$$

$$\text{MSE} = \frac{1}{N} \sum_{i=0}^{N-1} (x_i - y_i)^2$$

三个指标的作用分别是：

- 最大绝对误差：观察最差元素的偏差。
- MAE：观察平均误差水平。
- MSE：对较大的误差给予更高权重。

FP16 输出可能出现溢出。当前程序遇到非有限输出时，会记录数量，并将上述误差字段标为不可用，避免将有限子集的误差冒充完整结果。

2. 计算有效数据压缩率

设原始输入每个元素占 w 字节，FP32 时为 4，FP16 时为 2：

$$L_{\text{input}} = Nw$$

MXFP8 中，每个元素占一个字节，每个分组另有一个 scale 字节：

$$L_{\text{MXFP8}} = N + K$$

NVFP4 中，每两个元素占一个字节，每个分组一个 scale 字节，整个张量另有 4 字节 global scale：

$$L_{\text{NVFP4}} = \left\lceil \frac{N}{2} \right\rceil + K + 4$$

因此，有效数据压缩率为：

$$C_{\text{payload}} = \frac{Nw}{L}$$

对于完整分组、大规模 FP32 输入，MXFP8 的有效压缩率约为：

$$C_{\text{MXFP8}} \approx \frac{4}{1 + 1/32} = 3.8788$$

忽略 global scale 的固定开销，NVFP4 的有效压缩率约为：

$$C_{\text{NVFP4}} \approx \frac{4}{1/2 + 1/16} = 7.1111$$

因此，包含 scale 开销后的压缩率，不能直接写成固定的 4 倍或 8 倍。

3. 计算完整文件压缩率

当前输入文件头为 28 字节，低精度权重文件头为 72 字节。完整文件压缩率为：

$$C_{\text{file}} = \frac{28 + Nw}{72 + L_{\text{data}} + K}$$

其中，MXFP8 的 data 字节数等于元素数量，NVFP4 的 data 字节数等于元素数量除以二后向上取整。NVFP4 global scale 已经包含在 72 字节文件头中，因此完整文件压缩率的分母不再重复加 4。

FP32、FP16 和 BF16 输出

反量化先得到 FP32 中间值，再转换为指定输出类型：

$$y_i = \begin{cases} \hat{x}_i, & \text{FP32} \\ \text{RN}_{\text{FP16}}(\hat{x}_i), & \text{FP16} \\ \text{RN}_{\text{BF16}}(\hat{x}_i), & \text{BF16} \end{cases}$$

这里的 RN 使用 **round-to-nearest-even**，即距离相同时选择尾数最低保留位为偶数的候选值。

需要区分：

- 低精度 E4M3/E2M1 编码：中点选择较小幅值。
- 最后的 FP16/BF16 类型转换：使用 nearest-even。

三. 程序实现与使用方式

整体架构

本项目采用分层设计，将命令行交互、文件读写、设备运行管理、数值计算和正确性验证分别组织。正式实现集中在 `Core/` 目录中，用户通过 Python 命令行入口运行程序，主要量化与反量化计算由 C++/CUDA 实现。MXFP8 和 NVFP4 共用参数处理、文件管理、测试与性能统计等基础功能；两种格式各自维护对应的量化 kernel。项目同时支持 NVIDIA CUDA 和摩尔线程 MUSA 后端。

以下是核心模块 `Core/` 的清单：

模块	位置	主要职责
命令行入口	<code>Core/tools/quantize.py</code>	解析参数、选择格式与后端、组织输出
应用流程	<code>Core/app/</code>	串联输入读取、GPU 计算、CPU 校验与文件保存
文件读写	<code>Core/io/</code>	读取输入张量，保存和读取低精度权重及反量化张量
设备运行管理	<code>Core/runtime/</code>	管理显存、数据传输、kernel 启动和计时
CUDA 算子	<code>Core/kernels/</code>	实现编码、缩放、归约、量化和反量化
CPU 参考实现	<code>Core/reference/</code>	提供独立参考结果及冻结测试数据
测试与性能评估	<code>Core/tests/</code> 、 <code>Core/benchmarks/</code>	验证边界情况、正确性和性能
MUSA 适配	<code>Core/backends/musa/</code>	实现摩尔线程平台相关支持

数据处理流程

首先，程序读取输入路径、量化格式、缩放模式、舍入方式和输出精度等参数。参数可以来自 TOML 配置文件，也可以直接在命令行指定；显式命令行参数优先于配置文件。

```
python3 Core/tools/quantize.py run --input input/917/1.fp32 \
    --format mxfp8 --output-type fp16 --output-dir output/917 --save all
```

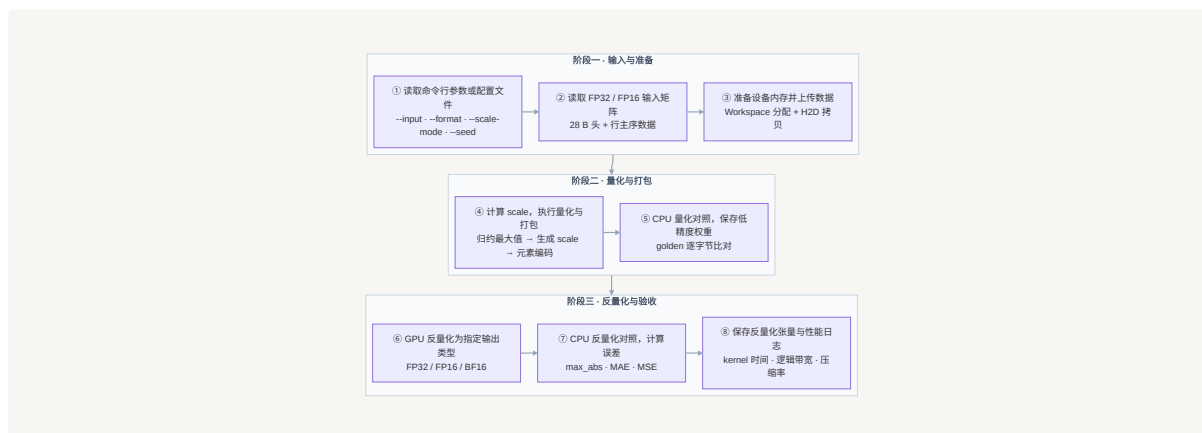
`--format` 选择 `mxfp8/nvfp4`，
 `--output-type` 选择 `fp32/fp16/bf16`，
 `--output-dir` 指定实际保存目录，文件名为 `<输入名>\_<格式>\_\_<入>\_<出>`
 `--save` 默认 `all`，可改为 `weights`、`tensor`、`log` 或组合，

随后，程序读取输入文件头，检查数据类型、矩阵形状和数据长度。FP32 输入直接读取，FP16 输入在主机端无损扩展为 FP32，使两种输入共用后续量化计算路径。输入上传至 GPU 后，程序根据格式与配置选择计算路径，生成低精度元素编码和 scale。量化结果下载后与 CPU reference 比较，并写入低精度权重文件。

最后，反量化阶段在 GPU 上读取低精度编码及 scale，恢复数值并转换为用户指定的 FP32、FP16 或 BF16。程序对反量化结果进行 CPU 对照，再计算原始输入与最终输出之间的误差，生成结果文件和日志。

CPU 对照用于判断 GPU 实现是否遵循数值规则；误差统计用于衡量低精度表示带来的信息损失，两者用途不同

下面是简易的程序流程：



核心计算模块

这一节讲如何分配 GPU 工作。公式已在第二章写过，这里不用重复推导。

MXFP8 量化

对于 `block + nearest` 配置，MXFP8 使用融合的向量化量化 kernel。程序将分组最大绝对值归约、scale 计算、元素编码和结果写回组织在同一个 kernel 中，以减少中间数据读写和 kernel 启动开销。向量化路径使用 float4 加载连续输入，由 8 个线程共同处理一个包含 32 个元素的量化分组。编码后，将四个 E4M3 字节合并为一次 32 位写回。对于不足向量宽度的尾部数据，采用边界判断和标量处理。nearest 编码根据 FP32 的指数与尾数直接计算 E4M3 编码，避免逐个枚举候选数值。其他配置使用相应的通用路径。

以下是 mxfp8_quantize_vectorized_kernel 程序介绍

1. 向量化读取

```
constexpr unsigned kLanesPerGroup = 8;
const std::size_t item = blockIdx.x * 256ull + threadIdx.x;
const std::size_t group = item / kLanesPerGroup;
const unsigned lane = threadIdx.x & 31;
const unsigned sublane = lane & (kLanesPerGroup - 1);

// 每个线程负责当前量化分组中的四个连续元素。
const std::size_t base =
    group * 32ull + static_cast<std::size_t>(sublane) * 4ull;

float values[4] = {0.0f, 0.0f, 0.0f, 0.0f};

if (base + 3 < n) {
    const float4 v =
        *reinterpret_cast<const float4*>(input + base);
    values[0] = v.x;
    values[1] = v.y;
    values[2] = v.z;
    values[3] = v.w;
} else {
    // 尾部不足四个元素时，仅读取有效位置。
    for (unsigned j = 0; j < 4; ++j)
        if (base + j < n)
            values[j] = input[base + j];
}
```

完整的四元素片段使用 `float4` 加载；尾部不足四个元素时回退到标量访问。无效位置保持为零，可以参与最大绝对值归约而不改变结果。

2. 组内归约、scale 计算与广播

```
float maximum = 0.0f;
// 先计算当前线程负责的四个元素的最大绝对值。
for (unsigned j = 0; j < 4; ++j)
    maximum = fmaxf(maximum, fabsf(values[j]));

// 再在八线程子组内完成归约。
for (unsigned step = 4; step; step >=> 1) {
    maximum = fmaxf(
        maximum,
        __shfl_down_sync(
            0xffffffffu, maximum, step, kLanesPerGroup));
}

unsigned code = 127;
```

```

if (sublane == 0 && base < n) {
    code = scale_code(maximum, 1.0f);
    scales[group] = static_cast<std::uint8_t>(code);
}

// 每组只计算一次 scale 编码，再广播给其他线程。
code = __shfl_sync(0xffffffffu, code, 0, kLanesPerGroup);
const float scale =
    scalbnf(1.0f, static_cast<int>(code) - 127);

```

每个线程先计算自身四个元素的最大绝对值，再通过 shuffle 在 8 个线程之间归约。子组首线程计算并保存 E8M0 scale 编码，随后将编码广播给组内其他线程。

由于输入值仍保留在寄存器中，后续编码可以直接复用，无需再次读取输入数组。

3. 编码合并并写回

```

std::uint32_t packed = 0;
for (unsigned j = 0; j < 4; ++j) {
    if (base + j < n) {
        packed |= static_cast<std::uint32_t>(
            encode_e4m3_nearest(values[j] / scale))
            << (8 * j);
    }
}

if (base + 3 < n) {
    // 四个 E4M3 字节合并为一次 32 位写回。
    *reinterpret_cast<std::uint32_t*>(data + base) = packed;
} else {
    // 避免尾部 32 位存储越过输出缓冲区。
    for (unsigned j = 0; j < 4 && base + j < n; ++j)
        data[base + j] =
            static_cast<std::uint8_t>(packed >> (8 * j));
}

```

这里将四个 8 bit 编码合并为一个 32 bit 数进行写回。这只是合并访存操作，每个 MXFP8 元素仍占一个字节，并没有改变数据格式。

NVFP4 量化

NVFP4 需要整个张量共享的 global scale，因此量化流程先执行全局最大绝对值归约，再执行局部分组量化。全局归约采用两级结构：第一阶段由多个线程块分别计算局部最大值，第二阶段合并这些结果并计算 global scale。

对于 `block + nearest` 配置，局部量化 kernel 将分组最大值归约、E4M3 block scale 编码、E2M1 元素编码和四位打包融合执行。每个线程负责两个相邻元素，并生成一个 packed 字节，从而满足每个字

节保存两个四位元素的要求。对奇数长度输入，最后一个字节只保存一个有效元素，高四位补零；所有输入访问和结果写回均进行尾部边界判断。

以下是nvfp4相关kernel介绍

1. 全局最大值归约

第一阶段由多个 CUDA thread block 分别计算局部最大值：

```
__global__ void maximum(
    const float* input, float* partial, std::size_t n) {

    __shared__ float s[kReductionShared];
    float v = 0;

    // grid-stride loop: 线程跨步处理多个输入位置。
    for (std::size_t i = blockIdx.x * 256ull + threadIdx.x;
        i < n;
        i += gridDim.x * 256ull) {
        v = fmaxf(v, fabsf(input[i]));
    }

    v = block_maximum(v, s);

    if (threadIdx.x == 0)
        partial[blockIdx.x] = v;
}
```

第二阶段由一个线程块合并 `partial`，计算 global scale：

```
__global__ void finalize_max(
    const float* partial, float* state, unsigned n) {

    __shared__ float s[kReductionShared];
    float v = 0;

    for (unsigned i = threadIdx.x; i < n; i += 256)
        v = fmaxf(v, partial[i]);

    v = block_maximum(v, s);

    if (threadIdx.x == 0) {
        state[0] = v;
        state[1] = nvfp4 && v > 0
            ? fmaxf(divide_rn(v, 2688.0f), __int_as_float(1))
            : 1.0f;
    }
```

```
}
}
```

其中：

- `state[0]` 保存整个张量的最大绝对值。
- `state[1]` 保存 global scale。
- `2688 = 6 × 448`，对应 E2M1 元素和 E4M3 block scale 的最大幅值乘积。
- `__int_as_float(1)` 表示最小正 FP32 非正规数，用于避免 global scale 下溢为零。

2. 局部分组归约与 scale 编码

每个线程处理两个相邻元素，8 个线程共同处理一个包含 16 个元素的量化分组：

```
const std::size_t item = blockIdx.x * 256ull + threadIdx.x;
const std::size_t i = item * 2;
const unsigned lane = threadIdx.x & 31;
constexpr unsigned kLanesPerGroup = kBlockSize / 2;

const float low_value = i < n ? input[i] : 0.0f;
const float high_value = i + 1 < n ? input[i + 1] : 0.0f;

float maximum = fmaxf(fabsf(low_value), fabsf(high_value));

for (unsigned step = kLanesPerGroup / 2; step; step >= 1) {
    maximum = fmaxf(
        maximum,
        __shfl_down_sync(
            0xffffffffu, maximum, step, kLanesPerGroup));
}

unsigned code = 0;
if (lane % kLanesPerGroup == 0 && i < n) {
    code = scale_code(maximum, state[1]);
    scales[i / kBlockSize] = code;
}

code = __shfl_sync(0xffffffffu, code, 0, kLanesPerGroup);
```

组内归约得到最大绝对值后，首线程结合 global scale 计算 E4M3 block scale 编码。编码保存在 scale 数组中，并广播给其他线程。

这里的 `code` 是 **block scale** 的编码，不是元素的 E2M1 编码。

3. 元素编码与四位打包

```

if (i < n) {
    // 将 global scale 与解码后的 block scale 合成有效 scale。
    const float scale =
        state[1] * decode_e4m3(static_cast<std::uint8_t>(code));

    const unsigned low = encode(
        scale > 0 ? low_value / scale : 0,
        true, false, 0, 0, false);

    const unsigned high = i + 1 < n
        ? encode(
            scale > 0 ? high_value / scale : 0,
            true, false, 0, 0, false)
        : 0;

    // 偶数元素低四位，奇数元素高四位。
    data[item] =
        static_cast<std::uint8_t>(low | (high << 4));
}

```

`encode` 调用中的 `true, false` 分别表示采用 E2M1 编码、关闭随机舍入，即当前的 nearest 路径。每个线程独占一个输出字节，避免两个线程分别修改同一个字节所带来的写入冲突。若输入长度为奇数，最后一个字节的高四位设为零。`scale > 0` 的判断处理零 scale 情况，避免除零。用于元素归一化的是 **global scale 与实际解码后的 block scale 的乘积**。

反量化与类型转换

两种格式共用反量化模板，每个线程负责一个输出元素。模板参数 `T` 决定最终输出为 FP32、FP16 还是 BF16。

这段代码包含三个关键步骤：

1. **读取编码**。MXFP8 读取一个字节；NVFP4 根据元素下标选择字节的低四位或高四位。
2. **恢复数值**。解码元素，读取对应 scale，再进行 FP32 乘法。
3. **转换精度**。根据模板类型输出 FP32、FP16 或 BF16。

```

template <typename T>
__global__ void dequantize_kernel(
    const std::uint8_t* data,
    const std::uint8_t* scales,
    const float* state,
    T* output,
    std::size_t n,
    std::size_t group) {

    const auto i = blockIdx.x * 256ull + threadIdx.x;
    if (i >= n) return;

```

```

// NVFP4 提取对应四位；MXFP8 直接读取一个字节。
const unsigned code = nvfp4
    ? ((data[i / 2] >> (4 * (i % 2))) & 15)
    : data[i];

// 将低精度编码解码为 FP32 数值。
const float value = nvfp4
    ? ((code & 8)
        ? -magnitude2(code & 7)
        : magnitude2(code & 7))
    : decode_e4m3(code);

// 恢复 scale、执行乘法，再转换为指定输出精度。
output[i] = cuda_output::Format<T>::convert(
    multiply_rn(
        value,
        effective(
            scales, state,
            group == kBlockSize ? i / kBlockSize : 0)));
}

```

四. 正确性验证与误差评估

验证目标与方法

本项目将验证分为实现正确性和量化精度两个层次：

- 实现正确性检查 GPU 是否按照既定的编码、缩放、舍入和反量化规则执行。项目使用 CPUreference 生成参考结果，将 GPU 输出与参考结果进行比较。
- 量化精度检查原始输入与最终反量化输出之间的差异，用于衡量低精度表示造成的信息损失。

两类检查的区别如下：

验证内容	比较对象	目的
量化实现正确性	GPU 编码与 CPU 编码	检查 packed data、scale 等是否一致
反量化实现正确性	GPU 输出与 CPU 参考输出	检查解包、解码、缩放和类型转换
量化误差评估	原始输入与最终反量化输出	衡量低精度表示的精度损失

CPU reference 与比较规则

量化结果比较

CPU reference 保留易于核查的枚举式编码方法，不复用 GPU 的快速编码器。GPU 可以采用直接编码、向量化访问和融合 kernel，但必须产生符合相同规则的结果。

量化结果主要检查：

- 矩阵形状和量化分组大小是否一致。
- packed data 是否逐字节一致。
- scale 数组是否逐字节一致。
- NVFP4 global scale 的 FP32 位模式是否一致。

代表代码如下：

```
if (a.rows != b.rows ||
    a.cols != b.cols ||
    a.group != b.group ||
    a.data != b.data ||
    a.scales != b.scales ||
    std::memcmp(&a.global, &b.global, 4)) {
    throw std::runtime_error("pipeline CPU/GPU packed mismatch");
}
```

这种比较不仅能发现数值偏差，也能发现打包位置错误、scale 分组错误和尾部编码错误。

反量化结果比较

CPU 首先恢复 FP32 参考值，再根据目标精度生成用于比较的结果。
不同输出类型采用以下规则：

输出类型	比较规则
FP32	有限值绝对差不超过 10^{-6} ，同时检查零的符号
FP16	CPU 参考值转换为 FP16 后，逐位比较
BF16	CPU 参考值转换为 BF16 后，逐位比较

边界条件与功能测试

仅测试普通随机矩阵不足以验证 kernel 的正确性。本项目进一步覆盖分组边界、线程块边界、数值边界和文件异常情况。

1. 测试覆盖范围

类别	代表测试	主要检查内容
输入长度	0~65、255/256/257、1023/1024/1025	空输入、尾部处理和边界访问
零值	全零、负零	零 scale 分支及符号规则
极端有限值	极小数、最小正非正规数、较大有限值	scale 下溢、饱和和数值稳定性
舍入边界	相邻可表示值的中点及两侧	nearest 中点规则
随机舍入	固定 seed、不同 seed、中点样本	可复现性和随机路径有效性
输入输出类型	两种输入、三种输出	类型转换与数值一致性
文件协议	错误 magic、错误版本、截断及长度不匹配	非法文件拒绝
特殊输入	NaN、正负无穷大	输入合法性检查

类别	代表测试	主要检查内容
FP16 输出溢出	大幅值输入	非有限输出计数及误差日志
独立反量化	保存 <code>.lpq</code> 后重新加载	文件往返和恢复结果一致性

2. 输出边界检查

测试在输出缓冲区前后设置固定哨兵值，执行 kernel 后检查哨兵是否被修改。

检查对象包括：

- 低精度元素数据。
- scale 数组。
- FP32、FP16、BF16 反量化输出。

该方法用于发现已覆盖用例中的越界写入。它不能替代专业内存检查工具对越界读取、未初始化访问和数据竞争的检查。

3. 当前测试结果

本次本地 CUDA 完整测试结果为：

```
jky@JKY:~/Learning-CUDA-project/02_quant_dequant/YeKcaJ$ ctest --test-dir Core/build --output-on-failure
Internal ctest changing into directory: /home/jky/Learning-CUDA-project/02_quant_dequant/YeKcaJ/Core/build
Test project /home/jky/Learning-CUDA-project/02_quant_dequant/YeKcaJ/Core/build
  Start 1: core_tools
1/9 Test #1: core_tools ..... Passed    2.01 sec
  Start 2: core_reduction
2/9 Test #2: core_reduction ..... Passed    0.63 sec
  Start 3: core_device_math
3/9 Test #3: core_device_math ..... Passed    0.22 sec
  Start 4: mxfp8_pipeline_io
4/9 Test #4: mxfp8_pipeline_io ..... Passed   11.01 sec
  Start 5: mxfp8_pipeline_regression
5/9 Test #5: mxfp8_pipeline_regression ..... Passed    0.94 sec
  Start 6: mxfp8_frozen_hashes
6/9 Test #6: mxfp8_frozen_hashes ..... Passed    0.01 sec
  Start 7: nvfp4_pipeline_io
7/9 Test #7: nvfp4_pipeline_io ..... Passed   10.74 sec
  Start 8: nvfp4_pipeline_regression
8/9 Test #8: nvfp4_pipeline_regression ..... Passed    0.66 sec
  Start 9: nvfp4_frozen_hashes
9/9 Test #9: nvfp4_frozen_hashes ..... Passed    0.01 sec

100% tests passed, 0 tests failed out of 9

Total Test time (real) = 26.24 sec
```

注意：26.24 秒是整套测试的运行时间，不是算子的性能耗时，也不代表 144 组误差评估结果。

固定误差评估方案

输入数据

误差评估使用固定输入，避免在比较格式和配置时更换数据。

项目	设置
矩阵形状	128 × 129
元素数量	16,512
生成 seed	1234
输入类型	FP32、FP16

项目	设置
数据分布	uniform、normal、outlier
数据校验	manifest 中记录 SHA256

三类数据的具体定义为：

分布	生成方法	观察目的
uniform	均匀分布 $U(-3, 3)$	一般有限范围内的量化误差
normal	标准正态分布 $N(0, 1)$	集中分布数据上的误差
outlier	正态数据，首元素改为 +1000，末元素改为 -1000	离群值对缩放与精度的影响

输入位于 `input/evaluation-v1/`。FP16 数据先转换为实际 FP16 值再写入文件，因此 FP16 评估比较的是文件中实际保存的输入，不是转换前的高精度生成值。

配置组合

完整评估覆盖：

$$3 \times 2 \times 2 \times 2 \times 2 \times 3 = 144$$

六个维度依次为：

维度	取值
数据分布	uniform、normal、outlier
输入类型	FP32、FP16
量化格式	MXFP8、NVFP4
缩放方式	block、tensor
舍入方式	nearest、stochastic
输出类型	FP32、FP16、BF16

通过一下指令进行评估

```
python3 Core/tools/quantize.py evaluate --backend cuda
```

三类矩阵的量化误差

为便于比较，本节固定以下条件：

FP32 输入、输出、block scaling、nearest 舍入，MXFP8 分组大小为 32，NVFP4 分组大小为 16。

1. 实测结果

格式	分布	最大绝对误差	MAE	MSE
MXFP8	uniform	0.124997	0.0346832	0.00220622
MXFP8	normal	0.201626	0.0180060	0.000710648
MXFP8	outlier	24.000000	0.0209094	0.0704779
NVFP4	uniform	0.499591	0.133239	0.0309889

格式	分布	最大绝对误差	MAE	MSE
NVFP4	normal	0.483356	0.0718482	0.00929973
NVFP4	outlier	2.197041	0.0728342	0.0104780

2. 结果分析

- 在 uniform 和 normal 两类输入中，MXFP8 的最大绝对误差、MAE 和 MSE 均低于 NVFP4。这与两种格式的表示能力相符：E4M3 使用 8 bit，具有更多可表示数值；E2M1 使用 4 bit，可表示幅值更稀疏，因此通常以更大的误差换取更高的压缩率。这一结论限于本节的测试数据和配置，不代表任意输入上每一个误差指标都具有相同排序。
- outlier 数据出现了一个值得关注的现象：NVFP4 的最大绝对误差和 MSE 小于 MXFP8，但 MAE 大于 MXFP8。检查实际反量化文件发现，两个极端值的恢复结果为：

原始值	MXFP8 恢复值	NVFP4 恢复值
+1000	+1024	+1000
-1000	-1024	-1000

对于 MXFP8，包含异常值的分组选择 $scale=4$ 。1000 归一化后得到 250，nearest E4M3 编码选择 256，最终恢复为 1024，因此产生 24 的绝对误差。

NVFP4 的两级缩放在该组数据上恰好使极端值恢复为 ± 1000 ，减少了少数极端位置对最大误差和 MSE 的影响。但其余普通元素的平均误差仍然更大，因此 MAE 高于 MXFP8。

这说明：**不能只根据最大误差或 MSE 判断一种格式整体更精确，应结合 MAE、数据分布和缩放方式共同分析。**

Block 与 tensor 缩放对比

本节继续固定 FP32 输入、FP32 输出和 nearest 舍入，仅改变缩放方式。

1. 实测 MAE 对比

格式	分布	block MAE	tensor MAE
MXFP8	uniform	0.0346832	0.0346832
MXFP8	normal	0.0180060	0.0180060
MXFP8	outlier	0.0209094	0.0209756
NVFP4	uniform	0.133239	0.146702
NVFP4	normal	0.0718482	0.101937
NVFP4	outlier	0.0728342	0.800046

2. 结果分析

对于 NVFP4，block 模式在这三类数据上均取得更低 MAE，尤其在 outlier 数据上差异明显。

tensor 模式中，整个张量共享一组缩放参数。 ± 1000 的异常值扩大了全张量的表示范围，使大量幅值接近 1 的普通元素在归一化后靠近零，容易被 E2M1 编码为零。

block 模式允许不同分组使用不同的局部 $scale$ ，使异常值对局部 $scale$ 的直接影响主要集中在其所在分组。其他分组可以继续按照自身范围进行量化。不过，NVFP4 的 global $scale$ 仍由整个张量决定，不能认为各组完全独立。

MXFP8 在普通分布上的两种模式误差非常接近。这与其 2 的幂缩放及 E4M3 浮点表示有关：在未触及非正规数、下溢或饱和边界时，改变 2 的幂 scale 不一定改变最终舍入网格。因此，**block scaling 并不保证每种格式、每种数据上的误差都明显降低。**

压缩率评估

对于同一形状、同一输入类型和缩放模式，压缩率由文件布局决定，不随 uniform、normal 或 outlier 分布改变。

本节采用 FP32 输入，共 16,512 个元素：

格式	缩放方式	有效数据压缩率	完整文件压缩率
MXFP8	block	3.878788×	3.864094×
MXFP8	tensor	3.999758×	3.984082×
NVFP4	block	7.108050×	7.059402×
NVFP4	tensor	7.995158×	7.933245×

有效数据压缩率包含元素编码和 scale 的必要开销；完整文件压缩率进一步包含文件头。

block 模式需要为多个分组保存 scale，因此压缩率略低于 tensor 模式。但前面的误差结果表明，较少的 scale 存储开销可能伴随更大的量化误差。

因此，缩放方式的选择是在**存储开销和数值精度之间进行权衡**，不能只根据压缩率判断优劣。

本章小结

本项目通过 CPU reference 对照、冻结数据校验、边界测试和文件协议测试，验证了量化、打包、解包及三种输出精度的实现。当前本地 CUDA 完整 CTest 9/9 通过，保存的 144 组评估记录均通过 CPU 量化和反量化对照。误差评估表明，在本次普通分布测试中，MXFP8 具有更低的量化误差，NVFP4 则提供更高的压缩率。含异常值时，缩放策略和少数极端元素会显著影响误差指标，需要结合最大绝对误差、MAE 和 MSE 分析，不能仅使用单个指标概括整体精度。

本章只讨论正确性、量化误差和压缩率；kernel 时间、带宽及优化收益在下一章单独分析。

本章数据相关路径：

- 完整误差评估结果：[Learning-CUDA-project/02_quant_dequant/1e1e1e1/output/evaluation-v1/cuda/summary.json](https://github.com/learning-cuda-project/02_quant_dequant/commit/1e1e1e1/output/evaluation-v1/cuda/summary.json)
- 固定输入参数与哈希：[Learning-CUDA-project/02_quant_dequant/1e1e1e1/input/evaluation-v1/manifest.json](https://github.com/learning-cuda-project/02_quant_dequant/commit/1e1e1e1/input/evaluation-v1/manifest.json)

五. 性能测试与优化分析

测试环境

本章主要分析 NVIDIA CUDA 平台，国产平台适配结果在后续章节单独介绍。

以下是NVIDIA CUDA平台使用的设备

项目	配置
GPU	NVIDIA GeForce RTX 3060 Laptop GPU
显存	6 GB
CPU	Intel Core i7-11800H

项目	配置
运行环境	WSL/Linux
NVIDIA 驱动	596.08
CUDA 编译器	CUDA 12.0, nvcc V12.0.140
输入类型	FP32
缩放方式	block
舍入方式	nearest
量化分组大小	MXFP8: 32; NVFP4: 16

1. 固定输入与测试规模

正式性能测试使用冻结的 FP32 正态分布输入，形状为 $1 \times N$ ，生成种子为 20260909。

简写	实际元素数	FP32 数据大小，不含文件头
1M	1,048,576	4 MiB
4M	4,194,304	16 MiB
16M	16,777,216	64 MiB

输入保存在 `input/benchmark-v1/`，测试前后均校验 SHA256，保证使用同一批实际数据。

该输入与第四章的 `evaluation-v1` 不同：前者用于性能测试，后者用于多分布、多配置误差评估。

2. 计时范围

项目区分设备驻留计算和主机调用两种计时范围。

范围	包含内容	用途
<code>resident_gpu</code>	GPU 量化序列或反量化 kernel	比较计算路径的性能
<code>host_api</code>	量化调用中的分配、上传、计算、下载及释放	观察主机调用开销

`resident_gpu` 使用 CUDA event 计时，显存提前分配并复用，不包含文件读写、CPU 校验和主机与设备之间的数据传输。

NVFP4 的量化时间包含全局归约和局部量化，不能只取最后一个融合 kernel 的耗时作为完整量化时间。

每组 GPU 测试预热 3 次，正式测量 20 次：

- median：排序后中间两个样本的平均值。
- P95：使用 nearest-rank 方法，20 个样本中取排序后的第 19 个。
- CPU reference：仅测试 1M，量化预热一次后计时三次。

当前测试未锁定 GPU 频率，因此仍可能受到功耗、温度及系统负载的影响。

性能指标与比较规则

耗时与加速比

量化和反量化分别记录耗时，不能混为同一个算子的执行时间。

相对基线加速比定义为：

$$S_{\text{baseline}} = \frac{T_{\text{baseline}}}{T_{\text{current}}}$$

相对上一轮加速比定义为：

$$S_{\text{previous}} = \frac{T_{\text{previous}}}{T_{\text{current}}}$$

其中，时间均使用同格式、同元素数、同计时范围的 median。

历史基线取第 0 次保存的 `quant_optimized / resident_gpu`

有效逻辑带宽

程序根据输入与输出的逻辑字节数计算带宽：

$$BW_{\text{logical}} = \frac{L_{\text{read}} + L_{\text{write}}}{T_{\text{ms}} \times 10^6} \text{ GB/s}$$

FP32 输入的量化逻辑数据量为：

$$L_{\text{quant}} = 4N + L_{\text{packed}}$$

反量化逻辑数据量为：

$$L_{\text{dequant}} = L_{\text{packed}} + w_{\text{out}}N$$

其中，低精度载荷包含元素编码及必要 scale，输出元素字节数分别为 FP32 的 4 字节或 FP16/BF16 的 2 字节。

逻辑带宽不是硬件实测 DRAM 带宽。 例如，NVFP4 全局归约会额外读取输入并访问中间结果，这些内部流量没有全部计入逻辑字节数。

MXFP8 优化过程

1. 直接编码与 scale 融合

原编码路径存在查找和重复解码操作。针对 nearest 舍入，优化后直接提取 FP32 的指数和尾数，计算 E4M3 编码，减少查找和重复计算；随机舍入保留原有路径。随后，将组内最大值归约、scale 计算和元素编码融合到同一个 kernel 中，复用已经加载的输入值，减少中间数据访问和 kernel 启动。

元素数	第0次基线/ms	仅直接编码/ms	编码与scale融合/ms	融合后相对基线
1M	0.123904	0.055728	0.034816	3.56×
4M	0.467968	0.199680	0.123312	3.79×
16M	1.634304	0.708608	0.435648	3.75×

结果表明，直接编码与融合均带来了收益。该轮最终加速比反映两项改动的合并效果，不能全部归因于其中一项。

2. 向量化加载与合并写回

在融合版本基础上，进一步使用 float4 加载，使每个线程处理 4 个元素，8 个线程共同处理一个 32 元素分组。编码后的四个 E4M3 字节合并为一次 32 位写回，尾部采用标量处理。

元素数	上一轮/ms	向量化后/ms	相对上一轮	相对基线
1M	0.034816	0.025600	1.36×	4.84×
4M	0.123312	0.070656	1.75×	6.62×
16M	0.435648	0.269824	1.61×	6.06×

向量化减少了处理相同数据所需的线程数量及相关访存、地址计算指令，但并未改变元素的存储位宽和量化公式。该轮 1M 的 P95 为 0.094080 ms，明显高于 median，说明小规模测试存在较大波动，不能仅凭中位数认为所有调用均具有相同耗时。

NVFP4 优化过程

1. block scale 直接编码与局部量化融合

NVFP4 使用 E4M3 保存 block scale，因此可以复用直接编码方法减少 scale 编码开销。局部量化阶段进一步融合分组归约、scale 编码、E2M1 编码和四位打包，每个线程负责两个元素并写出一个字节。全局最大值归约仍独立保留。

元素数	第0次基线/ms	本轮/ms	相对上一轮	相对基线
1M	0.142336	0.063488	2.24×	2.24×
4M	0.450944	0.142336	3.17×	3.17×
16M	1.742800	0.519136	3.36×	3.36×

该轮记录的是 scale 直接编码和局部融合完成后的整体收益。

2. 全局归约减少同步

全局归约改为先在 warp 内使用 shuffle，再通过少量共享内存合并各 warp 的结果，减少整块同步和共享内存访问。

元素数	上一轮/ms	本轮/ms	相对上一轮	相对基线
1M	0.063488	0.048128	1.32×	2.96×
4M	0.142336	0.140288	1.01×	3.21×
16M	0.519136	0.518144	1.00×	3.36×

改善主要体现在 1M。对于更大规模，整体耗时还受到输入扫描、后续量化等阶段影响，因此减少同步没有带来同比例的总时间下降。

3. partial 数量调优

进一步将全局归约的 partial block 上限从 4096 调整为 1024，并沿用 grid-stride loop 扫描输入。

元素数	上一轮/ms	本轮/ms	相对上一轮	相对基线
1M	0.048128	0.041344	1.16×	3.44×
4M	0.140288	0.140224	1.00×	3.22×

元素数	上一轮/ms	本轮/ms	相对上一轮	相对基线
16M	0.518144	0.518656	1.00×	3.36×

1M 获得进一步改善，4M 和 16M 基本不变。16M 的本轮时间还略高于上一轮，因此不应将该优化描述为“所有规模均明显加速”。

最终保留版本的性能

本节使用 `records/controlled-v1/cuda/` 保存的固定输入测试，作为最终保留实现的一次独立测量，不替换前述历史轮次数据。

1. 量化性能

格式	元素数	median/ms	P95/ms	逻辑带宽/GB/s
MXFP8	1M	0.021504	0.031744	245.333
MXFP8	4M	0.070656	0.075776	298.667
MXFP8	16M	0.269728	0.275456	312.946
NVFP4	1M	0.041984	0.090112	113.951
NVFP4	4M	0.139776	0.154368	136.908
NVFP4	16M	0.521216	0.522240	146.861

随着输入规模增大，两种格式的逻辑带宽均提高，反映固定启动开销得到更充分摊薄，设备利用率有所改善。虽然 NVFP4 输出更紧凑，但其量化需要全局归约和两级 scale 处理，因此不能仅凭输出位宽更低，就推断它一定比 MXFP8 量化更快，1M NVFP4 的 P95 明显高于 median，表明该批次存在尾部延迟波动。这里保留原始记录，不以其他批次的更好结果替换。

2. 反量化性能

下表使用同一批固定输入测试。时间为 median，括号内为逻辑带宽 GB/s。

格式	元素数	FP32: ms (GB/s)	FP16: ms (GB/s)	BF16: ms (GB/s)
MXFP8	1M	0.031744 (166.194)	0.025600 (124.160)	0.027648 (114.963)
MXFP8	4M	0.112640 (187.345)	0.103424 (122.931)	0.108544 (117.132)
MXFP8	16M	0.403392 (209.251)	0.363344 (139.966)	0.379904 (133.865)
NVFP4	1M	0.025600 (186.880)	0.020288 (132.442)	0.022528 (119.273)
NVFP4	4M	0.088112 (217.184)	0.078848 (136.312)	0.087040 (123.482)
NVFP4	16M	0.339968 (225.157)	0.303104 (141.838)	0.333824 (128.785)

FP16/BF16 每个输出元素占 2 字节，FP32 占 4 字节。因此，输出精度变化会同时影响写入量和类型转换工作。本次测试中，16 位输出耗时低于对应的 FP32 输出，但逻辑带宽并不一定更高，因为计算带宽时所使用的逻辑字节数也减小了。

主机调用与 workspace 复用

设备计算加速不等于整个调用过程获得相同比例的加速。主机调用还包含显存管理、数据传输和结果分配等开销。

下面列出最终固定输入测试中的 4M 结果：

格式	驻留GPU量化/ms	每次新建workspace的 host_api/ms	复用workspace的 host_api/ms
MXFP8	0.070656	5.697663	3.446899
NVFP4	0.139776	5.824294	3.537434

复用 workspace 后，显存和 event 的首次创建及最终销毁不进入重复调用计时，但仍包含上传、量化、下载以及主机结果分配和释放。结果说明，在当前测试条件下，主机调用开销显著大于设备驻留量化时间。继续改善实际应用性能时，应关注批量调用、缓冲区复用和减少传输，而不只是继续压缩 kernel 耗时。该结果针对**同一进程内的重复调用**，不能解释为每次重新启动 Python CLI 都会自动获得同样收益；它也不是包含文件 I/O、CPU 验证及反量化的完整程序端到端时间。

Nsight 工具辅助分析

工具分工

本项目使用两类工具分析不同层次的问题：

- **Nsight Systems**：观察 kernel 调用顺序、数据传输、分配及同步开销，识别主机调用和设备计算之间的关系。
- **Nsight Compute**：检查具体 kernel 的 DRAM 吞吐、SM 吞吐、占用率和寄存器等指标，辅助判断优化方向。

正式性能表采用未附加 profiler 的 CUDA event benchmark。Profiler 下的单次耗时用于诊断，不直接替代 benchmark 结果。

MXFP8 分析

现有 4M 输入的 NCU 分析记录中，向量化量化 kernel 的主要指标为：

指标	数值
Kernel duration	71.14 μ s
DRAM throughput	90.28%
SM throughput	81.95%
Achieved occupancy	88.92%
Registers/thread	25
Local memory spilling	0

该次采样显示，内存与计算资源均处于较高使用水平，DRAM 吞吐尤其突出，继续通过单一计算优化获得大幅加速的空间可能有限。较高占用率和未观察到寄存器溢出，也说明不能简单将增加 occupancy 作为首要目标。进一步优化可以检查写回访问效率和指令开销。

NVFP4 分析

现有 4M 输入的阶段性 NCU 记录为：

Kernel	DRAM throughput	SM throughput	Achieved occupancy	Duration
局部融合量化	56.08%	84.46%	88.33%	106.59 μ s
全局归约 maximum	88.01%	32.50%	88.55%	61.31 μ s

这组数据支持按阶段选择不同优化方向：

- 局部融合量化的计算侧压力更突出，应检查编码、除法和指令开销。
- 全局归约的访存压力更突出，应检查输入扫描与归约调度。

各版本相对历史基线的加速比

1. MXFP8

版本	1M	4M	16M
第0次：基线	1.00×	1.00×	1.00×
第1次中间步骤：直接编码	2.22×	2.34×	2.31×
第1次：直接编码与scale融合	3.56×	3.79×	3.75×
第4次：向量化	4.84×	6.62×	6.06×

2. NVFP4

版本	1M	4M	16M
第0次：基线	1.00×	1.00×	1.00×
第2次：scale编码与局部融合	2.24×	3.17×	3.36×
第3次：归约减少同步	2.96×	3.21×	3.36×
第6次：partial数量调优	3.44×	3.22×	3.36×

workspace 复用属于不同的主机调用计时范围，不纳入以上设备驻留量化加速比表。

本章小结

本项目围绕编码计算、kernel 融合、访存组织和归约调度进行了优化。MXFP8 通过 nearest 直接编码、scale 融合及向量化访问，在历史记录中相对基线取得约 4.84~6.62 倍的量化加速。NVFP4 通过 block scale 直接编码、局部融合、归约减少同步及 partial 数量调优，取得约 3.22~3.44 倍的加速。

最终固定输入测试中，16M 输入的 MXFP8 和 NVFP4 驻留 GPU 量化 median 分别为 0.269728 ms 和 0.521216 ms。

结果说明，软件模拟方案能够在不依赖原生 FP8/FP4 Tensor Core 的情况下完成低精度量化与反量化。

现阶段继续优化应区分设备计算和实际调用过程：设备侧根据各 kernel 的计算与访存压力选择方向，应用侧则优先考虑复用显存、批量处理和减少数据传输。当前结果可以作为项目交付性能，不将其表述为已经证明达到理论极限。

本章原始依据：

- Learning-CUDA-project/02_quant_dequant/YeKcaJ/OPTIMIZATION_LOG.md)
- Learning-CUDA-project/02_quant_dequant/YeKcaJ/records/controlled-v1/cuda/RESULTS.md)

- Learning-CUDA-project/02_quant_dequant/YeKcaJ/records/controlled-v1/cuda/environment.json)
- Learning-CUDA-project/02_quant_dequant/YeKcaJ/results/ncu/mxvp8-test-analysis.md)
- Learning-CUDA-project/02_quant_dequant/YeKcaJ/results/ncu/nvfp4-test-analysis.md)

六. 国产平台适配与验证

适配目标与实验环境

在完成 NVIDIA CUDA 平台的量化与反量化实现后，本项目进一步将程序迁移至摩尔线程 MUSA 平台，以验证低精度数值表示和计算流程的可移植性。适配过程中保持 MXFP8、NVFP4 的编码规则、缩放公式、舍入规则及文件格式不变，仅针对设备运行时、并行计算和浮点运算差异进行调整。

本次国产平台实验环境如下：

项目	配置
GPU	摩尔线程 MTT S4000
显存	48 GB
驱动与 MUSA SDK	5.1.0
编译器	mcc
目标架构	mp_22
输入类型	FP32、FP16
反量化输出类型	FP32、FP16、BF16

程序迁移方案

1. 公共逻辑与平台代码分离

为了避免为不同平台分别维护完整程序，本项目复用数值规则、文件读写和测试逻辑，将平台差异集中在后端实现中。

模块	处理方式
CPU reference 与 golden	两个平台共用
MXFP8、NVFP4 编码规则	保持一致
packed 权重文件与日志格式	保持一致
内存申请、数据传输、event 计时	映射到 MUSA 接口
量化 kernel	使用 MUSA 专用并行实现
反量化及其他公共设备代码	通过兼容接口和条件编译复用
编译与构建产物	CUDA、MUSA 分目录管理

主要适配代码集中在 `Core/backends/musa/`，其中 `quantize.cuh` 实现 MUSA 平台的融合量化kernel。运行时通过 `--backend musa` 选择后端，用户无需更换输入文件或重新学习一套操作方式。

2. 保持输出含义一致

适配后，程序仍输出题目要求的三类文件：

- 低精度权重文件：包含 header、scale 和 packed data。

- 反量化张量：按照指定的 FP32、FP16 或 BF16 类型保存。
- JSON 日志：记录误差、压缩率、量化与反量化耗时及逻辑带宽。

NVFP4 继续采用每字节保存两个 4-bit 元素的布局，不使用一个字节保存单个 4-bit 元素。

关键适配问题与解决方法

1. 线程子组与归约实现

CUDA 版本使用 warp shuffle 完成局部最大值归约，但这一实现不能仅根据接口名称直接迁移。在本次 MUSA 环境中，设备报告的 `warpSize` 为 128，而 SDK 同步 shuffle 接口采用 32-lane 逻辑子组。项目通过独立测试验证了 `width=4/8/16/32` 的行为，再据此实现局部归约。

优化后的 MUSA 量化 kernel 采用：

- `float4` 向量化加载；
- 子组 shuffle 最大值归约；
- scale 计算与元素编码融合；
- 合并写出量化数据。

纯 shuffle 的 scale 广播方案曾在零 scale 和极小值用例中出现回归失败，因此最终保留一次 shared memory 广播与同步。

这说明，平台迁移需要验证同步和通信语义，不能只替换 API 名称。

2. 浮点运算一致性

量化结果对缩放后的数值十分敏感。即使除法结果只相差一个很小的数值，也可能跨越舍入边界，导致最终编码与 CPU reference 不一致。

在本次环境中，普通 FP32 运算路径不能满足全部冻结测试要求；初期使用的严格除法实现又具有较高开销。

最终在 MUSA 分支中采用 FP64 中间结果，再以最近偶数舍入转换回 FP32：

```
__device__ inline float divide_rn(float a, float b) {
#ifdef LP_BACKEND_MUSA
    return __double2float_rn(
        static_cast<double>(a) / static_cast<double>(b));
#else
    return a / b;
#endif
}
```

乘法同样使用 FP64 中间结果，以处理普通乘法路径中的次正规数问题。

正确性验证

适配后继续使用同一套 CPU reference 和冻结测试，避免为新平台重新定义“正确答案”。

主要验证内容如下：

验证项目	内容与结果
CTest	保存的 MUSA 验证记录为 10/10 通过
packed data 与 scale	与 CPU reference 对照
反量化输出	覆盖 FP32、FP16、BF16
边界输入	覆盖空输入、尾部 block、零、负零及极端有限值
shuffle 语义	使用独立测试验证
误差评估	144 组配置全部通过 CPU 量化与反量化对照，无非有限输出

性能优化与测试结果

1. 测试口径

MUSA 性能测试采用 FP32 输入、`block + nearest`，分别测试 1M、4M 和 16M 元素。

每组预热 3 次、测量 20 次，使用 MUSA event 统计 GPU 驻留计算时间，以中位数作为主要性能指标。

其中：

- MXFP8 统计量化设备计算序列；
- NVFP4 还包含两级全局最大值归约；
- 不包含主机与设备间传输、显存分配、CPU 校验和文件读写。

1M 表示 1,048,576 个元素。

2. MUSA 优化过程

适配后的优化分为两轮：

第一轮迁移 shuffle 归约、向量化加载和合并写出；第二轮替换高开销严格除法实现，同时保持数值规则。

历史优化记录如下，单位为 ms：

格式	元素数	优化基线	第一轮：shuffle + 向量化	第二轮：除法替代	第二轮相对基线
MXFP8	1M	0.676629	0.505040	0.166549	4.06×
MXFP8	4M	2.564229	1.624251	0.399097	6.43×
MXFP8	16M	10.094629	6.327497	1.360480	7.42×
NVFP4	1M	0.622149	0.597931	0.238274	2.61×
NVFP4	4M	2.090983	1.747714	0.495760	4.22×
NVFP4	16M	8.095337	6.718057	1.647989	4.91×

第一轮降低了数据加载和线程通信开销；第二轮带来的提升更明显，说明浮点除法实现是初始 MUSA 版本的重要性能开销来源。

3. 固定输入下的跨平台比较

为保证最终跨平台比较使用相同数据，项目另外采用冻结 FP32 正态输入进行测试，随机种子为 `20260909`，并确认两端输入文件 SHA256 一致。

以下是这组独立测试中的量化中位耗时，单位为 ms：

格式	元素数	CUDA: RTX 3060 Laptop	MUSA: MTT S4000
MXFP8	1M	0.021504	0.172377
MXFP8	4M	0.070656	0.400114
MXFP8	16M	0.269728	1.363303
NVFP4	1M	0.041984	0.244434
NVFP4	4M	0.139776	0.506709
NVFP4	16M	0.521216	1.618926

本次测试中，CUDA 实现的量化耗时更低。但这些差异同时受到设备、编译器、数学运算实现和 kernel 映射方式影响，不能直接解释为两款硬件整体性能的差距。这组数据用于最终版本的跨平台比较，不与上一节历史优化数据混合计算优化加速比。

本章小结

本项目已在摩尔线程 MTT S4000 上完成 MXFP8、NVFP4 的真实设备适配，保留了原有数值规则、packed 文件格式和用户操作流程，并通过 CPU 对照与回归测试验证功能正确性。

适配过程中，线程子组语义和浮点运算行为是主要难点。通过平台专用 kernel、向量化加载、shuffle 归约及严格算术实现调整，改善了初始适配版本的性能。

当前仍存在以下限制：

- MUPTI 采集受到 MT-Perf 硬件连接问题阻断，尚未完成与 CUDA NCU 同等深度的性能分析。
- 尚未完成完整设备内存与竞争检查。
- 华为昇腾和沐曦平台尚未完成适配。

因此，本阶段的结论是：**完成了摩尔线程平台的功能适配和阶段性优化，但不能据此宣称已经达到该平台性能极限。**

七. 总结与展望

工作总结

本项目围绕 MXFP8 和 NVFP4 两种低精度格式，实现了从输入张量读取、量化编码、低精度权重保存，到反量化输出、误差统计与性能测试的完整程序，并完成了 NVIDIA CUDA 和摩尔线程 MUSA 平台的实现与验证。

主要工作包括以下四个方面：

1. 完成低精度量化与反量化功能

实现了 MXFP8 的 E4M3 元素编码与缩放，以及 NVFP4 的 E2M1 元素编码、局部与全局缩放、4-bit 打包和解包。其中，NVFP4 每个字节存储两个元素，满足 packed 存储要求。

程序支持 block-wise 和 per-tensor 缩放，提供 nearest 与 stochastic 舍入方式，支持 FP32、FP16 输入文件，以及 FP32、FP16、BF16 反量化输出。基础实现采用软件数值编码，不依赖硬件原生 FP8/FP4 Tensor Core。

程序通过统一命令行入口选择输入文件、量化格式、输出精度和保存目录，并生成低精度权重、反量化张量以及误差与性能日志。

2. 建立正确性验证体系

首先实现 CPU reference，并固定量化公式、文件格式和测试数据，作为 GPU 实现的对照依据。在此基础上，建立了编码与 scale 比较、反量化输出比较及边界测试，覆盖全零、负零、尾部 block、舍入边界和极端有限值等情况。同时，对均匀分布、正态分布和含异常值的数据开展误差评估。保存的验证记录显示，CUDA 的 9 项 CTest 和 MUSA 的 10 项 CTest 均通过；两平台各自的 144 组评估均通过 CPU 量化与反量化对照。这些结果表明，程序在已覆盖的测试范围内满足预期数值规则，但不能替代对全部可能输入的证明。

3. 完成性能分析与优化

CUDA 平台的优化主要包括直接编码、scale 与编码融合、warp shuffle 归约、向量化访存，以及全局归约参数调整。对于重复调用场景，另外实现了显存和 event 的复用，减少主机流程开销。

性能分析结合两类证据开展：使用 benchmark 衡量优化前后的耗时变化，使用 Nsight Systems 和 Nsight Compute 分析 kernel 执行及资源利用情况。

历史记录中，16M 元素的 MXFP8、NVFP4 量化相对各自固定优化基线分别取得约 **6.06×**、**3.36×** 的加速。由于历史轮次未完整保留实际输入哈希，这些结果作为同配置优化趋势；最终跨平台比较则单独使用冻结输入及哈希校验，避免混用测试口径。

4. 完成摩尔线程平台适配

在 MTT S4000 上实现了真实 MUSA kernel 执行，复用了公共数值定义、文件格式和 CPU reference，并针对线程子组语义及浮点运算差异进行适配。

通过 shuffle 归约、向量化加载及严格除法实现替换，改善了初始适配版本的性能。该过程说明，算法和数据格式可以跨平台复用，但并行实现与性能结论仍需要在目标平台重新验证。

实验结论

1. 压缩能力与数值精度需要权衡

低精度量化通过减少每个元素的存储位数降低数据体积，同时引入不可逆的数值误差。反量化虽然恢复为 FP32、FP16 或 BF16 类型，但不能恢复量化过程中丢失的信息。

在本项目的 FP32 输入、默认 block 配置及已测数据中，MXFP8 的有效数据压缩率约为 **3.88 倍**，NVFP4 约为 **7.11 倍**，均包含相应 scale 开销。

NVFP4 通常具有更高压缩率，但误差也更依赖输入分布和缩放方式。含异常值数据上的表现说明，不能仅依据位宽判断所有场景下的误差大小，还需要结合最大绝对误差、MAE 和 MSE 进行分析。

2. 优化应以实际瓶颈为依据

不同阶段的主要开销不同。直接编码减少数值转换计算，融合减少中间访存与启动开销，归约和向量化改善线程通信及访存效率。

随着计算开销下降，内存带宽可能逐渐成为主要限制。在本次 MXFP8 的 NCU 采样中，DRAM 吞吐利用率约为 **90.28%**，表明该测试条件下已具有较高的带宽利用率。但这一结果不足以证明所有输入规模、配置和设备均已达到性能极限。

因此，优化是否保留，应同时依据正确性回归和可重复的性能收益，而不是仅依据某种技术通常有效。

3. 跨平台适配不等于接口替换

不同平台的线程模型、编译器和浮点运算行为存在差异。同一优化方法迁移后，可能获得不同收益，也可能产生边界正确性问题。

本项目采用“先保证数值一致，再测量性能，最后进行平台专项优化”的顺序，使适配工作具有可检查的依据。

当前不足

目前项目仍存在以下不足：

1. **测试场景有限。** 性能评估主要围绕固定分布和 1M、4M、16M 三种规模展开，尚未充分覆盖真实模型权重及更多形状。
2. **部分路径尚未专项优化。** 主要性能优化集中在 FP32 内部计算、block 与 nearest 路径，不能将其收益直接推广到所有配置。
3. **FP16 输入仍需主机转换。** 当前先将 FP16 展开为 FP32，再送入量化 kernel，尚未实现直接读取 FP16 的专用量化路径。
4. **应用层收益尚未验证。** 当前衡量的是独立量化与反量化程序，没有接入真实模型推理，不能据此推断模型精度或端到端推理加速。

未来展望

后续项目会以以下几个方向继续开展：

1. **扩展真实数据与应用验证。** 引入真实模型权重，观察不同层、不同数值分布和异常值比例下的量化误差，并进一步评估模型精度、显存占用及端到端延迟。
2. **完善输入与计算路径。** 探索直接读取 FP16 的量化 kernel，减少主机转换和传输数据量；针对 stochastic、per-tensor 及不同规模输入开展独立分析。
3. **扩展国产平台支持。** 继续推进沐曦和华为昇腾平台适配，保持公共数值规则和文件格式一致。其中，昇腾平台需要结合 Ascend C 的计算与存储模型重新组织 kernel，不能直接沿用 CUDA 的线程实现。各平台均应通过同一套 CPU reference 和固定输入验证，再建立独立性能基线。

结语

最后，十分感谢启元实验室发起的InfiniTensor大模型与人工智能系统训练营，提供了一个能学习，能实战的平台，方向多元化，给大家更多的方向发展。通过项目的锤炼，使我学会了项目的主要成果不仅是量化与反量化 kernel，也包括统一的程序入口、输出文件、测试体系和性能记录。通过这一过程，进一步认识到：低精度计算需要同时考虑数值正确性、存储效率、硬件执行特点与工程可复现性，性能提升必须建立在可靠验证之上。如今正是ai最高潮之时，而我们aiifra-er正是当前风暴中ai巨轮的船基，让我们汲取前沿知识，拓宽自身视野，让国产ai行稳致远。